

CareerPilot

Your Agentic Career Co-pilot

Dependencies & Documentation

Setup Instructions, Project Goals & Dependency List

Codesprint 2026 — Powered by Poridhi.io

June 8, 2026

Contents

1	Project Goals	2
2	System Requirements	2
3	Environment Variables	2
3.1	Backend — /backend/.env	2
3.2	Frontend — /frontend/.env.local	3
4	Setup Instructions	3
4.1	1. Clone the Repository	3
4.2	2. Supabase Setup	3
4.3	3. Firebase Setup	4
4.4	4. Gemini API Keys	4
4.5	5. Groq & JSearch API Keys	5
4.6	6. Backend Setup	5
4.7	7. Frontend Setup	5
5	Backend Dependencies	6
6	Frontend Dependencies	6
7	API Reference	7
7.1	Authentication	7
7.2	Endpoints	7
8	Project Structure	8

Project Goals

CareerPilot aims to solve the fragmented job search experience by building a single AI-powered platform that:

1. **Knows the user** — Every feature is grounded in the user's actual CV via a RAG pipeline. No hallucinated generic profiles.
2. **Hunts jobs actively** — A Job Hunter Agent searches live listings via JSearch API, scores each against the user's CV, and explains the match reasoning with a two-pass fallback for sparse local markets.
3. **Identifies skill gaps** — The AI assistant compares the user's CV against job descriptions and benchmark profiles to surface missing skills with priority rankings.
4. **Drafts applications** — Personalized cover letters generated from actual CV experience, not templates.
5. **Builds learning roadmaps** — Week-by-week structured plans with resources and milestones tailored to the user's current gaps.
6. **Tracks progress** — Kanban board, goal setting, calendar reminders, and a progress dashboard keep the user accountable day-to-day.

System Requirements

Requirement	Version
Python	3.13 or higher
Node.js	18.0 or higher
npm	9.0 or higher
Git	Any recent version

Environment Variables

Backend — `/backend/.env`

```
# Supabase
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_KEY=your-service-role-key
SUPABASE_BUCKET=cvs

# Gemini
GEMINI_API_KEY=AIza... # embeddings, assistant, fit scoring

# Groq (feed recommendations)
GROQ_API_KEY=gsk...

# JSearch (job listings)
JSEARCH_API_KEY=ak...

# Firebase Admin SDK (choose one)
```

```
FIREBASE_CREDENTIALS_JSON={"type":"service_account",...}  
# OR  
FIREBASE_CREDENTIALS_PATH=firebase-credentials.json  
  
PORT=port_number  
FRONTEND_URL=http://localhost:frontend_port
```

Frontend — /frontend/.env.local

```
NEXT_PUBLIC_BACKEND_URL=http://localhost:backend_port  
  
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co  
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key  
  
NEXT_PUBLIC_FIREBASE_API_KEY=AIza...  
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=your-app.firebaseio.com  
NEXT_PUBLIC_FIREBASE_PROJECT_ID=your-project-id  
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=your-app.firebaseio.com  
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=your-sender-id  
NEXT_PUBLIC_FIREBASE_APP_ID=your-app-id
```

Setup Instructions

1. Clone the Repository

```
git clone https://github.com/SAfreen06/CareerPilot.git  
cd careerpilot
```

2. Supabase Setup

1. Create a new project at supabase.com.
2. Copy the **Project URL** and **service_role** key from Settings → API into your `.env`.
3. In the SQL editor, run the following migration:

```
-- Enable pgvector extension  
create extension if not exists vector;  
  
-- Users table  
create table public.users (  
  id uuid primary key default gen_random_uuid(),  
  firebase_uid text unique not null,  
  email text unique not null,  
  full_name text,  
  is_active boolean default true,  
  created_at timestamptz default now()  
);  
create index on public.users (firebase_uid);
```

```
-- CV embeddings table (768 dimensions)
create table public.cv_embeddings (
  id uuid primary key default gen_random_uuid(),
  source text not null,
  section text,
  candidate_id text,
  content text,
  embedding vector(768),
  created_at timestamptz default now()
);
create index on public.cv_embeddings
  using ivfflat (embedding vector_cosine_ops)
  with (lists = 100);

-- Similarity search RPC function
create or replace function match_cv_embeddings(
  query_embedding vector(768),
  match_count int,
  filter_source text default null
)
returns table (
  id uuid, content text, source text,
  section text, candidate_id text, distance float
)
language sql stable as $$
  select id, content, source, section, candidate_id,
    1 - (embedding <=> query_embedding) as distance
  from cv_embeddings
  where filter_source is null or source = filter_source
  order by embedding <=> query_embedding
  limit match_count;
$$;
```

3. Firebase Setup

1. Create a project at console.firebase.google.com.
2. Enable **Authentication** → **Google** sign-in provider.
3. Add `http://localhost:port` to **Authorized domains** under Authentication settings.
4. Go to **Project Settings** → **Service accounts** → **Generate new private key**.
5. Paste the JSON into `FIREBASE_CREDENTIALS_JSON` in `.env`, or save as `firebase-credentials.js` in `/backend`.
6. Copy your web app config values into `.env.local`.

4. Gemini API Keys

1. Go to aistudio.google.com/apikey.

2. Create **two separate API keys**.
3. Set one as `GEMINI_API_KEY` (embeddings, assistant, scoring).

Caution: The Gemini API key has rate limits. As it is used for various purposes (embeddings, assistant, scoring), the key may occasionally be exhausted. After the limit resets, it can be used again.

5. Groq & JSearch API Keys

1. Groq: sign up at console.groq.com and generate a key for `GROQ_API_KEY`.
2. JSearch: subscribe at [RapidAPI](https://rapidapi.com/jsearch) and copy the key for `JSEARCH_API_KEY`.

6. Backend Setup

```
cd backend
python -m venv .venv

# Activate virtual environment
source .venv/bin/activate # Linux / Mac
.venv\Scripts\activate # Windows

pip install -r requirements.txt
uvicorn app.main:app --reload
```

On successful startup you should see:

```
Firestore Admin SDK initialized successfully
INFO: Uvicorn running on http://127.0.0.1:8000
```

7. Frontend Setup

```
cd frontend
npm install
npm run dev
```

The app will be available at `http://localhost:port`.

Backend Dependencies

Package	Purpose
fastapi	Web framework and API routing
uvicorn	ASGI server for FastAPI
python-dotenv	Environment variable loading
pydantic[email]	Request/response schema validation
firebase-admin $\geq 6.3.0$	Firebase token verification
supabase	Supabase Python client
httpx	HTTP client (forced HTTP/1.1)
google-genai	Gemini embedding and generation
groq	Groq SDK for feed recommendations
python-multipart	Multipart file upload support
pdfplumber	PDF text extraction
python-docx	DOCX text extraction
sqlalchemy	ORM for model class definitions
apscheduler	Background scheduler for AI nudges
requests	HTTP calls to JSearch API

Full requirements.txt:

```
fastapi
uvicorn[standard]
python-dotenv
pydantic[email]
firebase-admin>=6.3.0
supabase
httpx
google-genai
groq
python-multipart
pdfplumber
python-docx
sqlalchemy
apscheduler
requests
```

Frontend Dependencies

Package	Purpose
next	React framework (App Router)
react / react-dom	UI rendering
typescript	Type safety
tailwindcss	Utility-first styling
firebase	Firebase Auth client SDK
lucide-react	Icon library

API Reference

Authentication

All protected endpoints require:

Authorization: Bearer <Firebase ID Token>

Endpoints

Endpoint	Method	Description
Users		
/api/users/me	POST	Sync Firebase user to Supabase on first login
/api/users/me	GET	Get current authenticated user profile
CV		
/api/cv/upload	POST	Upload CV file to Supabase Storage
/api/cv/ingest	POST	Parse, chunk, embed and index a CV
/api/cv/embedded-data	GET	Retrieve paginated CV sections by file ID
Assistant		
/api/assistant/	POST	Send a query to the AI career assistant
Jobs		
/api/jobs/health	GET	Health check for job hunter module
/api/jobs/test-raw	GET	Test JSearch raw response for a query
/api/jobs/test-cv-context	GET	Retrieve CV context preview for debugging
/api/jobs/test-scorer	GET	Test Gemini fit scorer with dummy data
/api/jobs/search	POST	Search live jobs and score them against CV
/api/jobs/feed	GET	Get three recommended job roles from Groq
/api/jobs/save	POST	Save a job to the Kanban tracker
Kanban		
/api/kanban/	GET	List all job applications for user
/api/kanban/	POST	Create a new job application
/api/kanban/{app_id}	PATCH	Update application status / notes
/api/kanban/{app_id}	DELETE	Delete an application
/api/kanban/cover-letter	POST	Generate a cover letter using Groq
/api/kanban/send-email	POST	Send application email via SMTP
Calendar		
/api/calendar/goals	GET	List goals with their tasks
/api/calendar/goals	POST	Create a new goal
/api/calendar/goals/{goal_id}	PATCH	Mark goal as completed (cascades to tasks)
/api/calendar/goals/{goal_id}	DELETE	Delete goal and its tasks

(continued on next page)

(continued from previous page)

Endpoint	Method	Description
/api/calendar/tasks	GET	List all tasks
/api/calendar/tasks	POST	Create a new task
/api/calendar/tasks/{task_id}/complete	PATCH	Mark a task as completed
/api/calendar/tasks/{task_id}	DELETE	Delete a single task
Nudges		
/api/nudges/generate	POST	Generate a new motivational nudge
/api/nudges/	GET	List all nudges for the user
/api/nudges/latest	GET	Get the most recent nudge (auto-marks seen)
/api/nudges/{nudge_id}/seen	PATCH	Mark a nudge as read
/api/nudges/{nudge_id}	DELETE	Delete a nudge
/api/nudges/context	GET	Debug: show context used for nudge generation
Progress		
/api/progress/stats	GET	Get application, task, and goal statistics

Project Structure

```

careerpilot/
+-- backend/
| +-- app/
| | +-- api/routes/
| | | +-- cv_ingest.py
| | | +-- assistant.py
| | | +-- users.py
| | | \-- jobs.py
| | +-- core/
| | | +-- config.py
| | | \-- firebase.py
| | +-- services/
| | | +-- assistant.py
| | | +-- cv_chunker.py
| | | +-- cv_parser.py
| | | +-- cv_structurer.py
| | | +-- embeddings.py
| | | +-- gemini.py
| | | +-- groq_client.py
| | | +-- jsearch.py
| | | \-- vector_store.py
| | +-- schemas/
| | +-- models.py
| | +-- db.py
| | \-- main.py
| +-- requirements.txt
| \-- .env
\-- frontend/

```

```
+-- app/  
+-- components/  
+-- lib/  
| +-- firebase.ts  
| \-- backend.ts  
+-- package.json  
\-- .env.local
```